

STOCKFLOW

Inventory Management System — Technical Case Study

Full-Stack Engineering Interview | Complete Solution

PART 1

Code Review & Debug
5 Issues Fixed

PART 2

Database Design
8 Tables Designed

PART 3

API Implementation
Full Endpoint Built

Issue Analysis Summary

The original endpoint has 5 distinct issues spanning security, reliability, and business logic. Below is the complete breakdown:

#	Issue	Production Impact	Fix	Severity
1	No Input Validation	Malformed requests crash the server; missing fields raise unhandled KeyError	Use request.json.get() with defaults; validate required fields and return 400 on failure	CRITICAL
2	No Error Handling / No Transaction	Two separate commits mean product can save but inventory fails — leaving orphaned records in DB	Wrap both inserts in a single try/except block with db.session.rollback() on failure	CRITICAL
3	No SKU Uniqueness Check	Duplicate SKUs silently overwrite data; two products share same identifier causing inventory confusion	Query for existing SKU before insert; return 409 Conflict if found	HIGH
4	No Auth/Authorization	Any unauthenticated caller can create products in any warehouse they don't own	Add @login_required decorator; verify warehouse belongs to caller's company	HIGH
5	Unsafe Price Handling	Floating point (e.g. 0.1+0.2 ≠ 0.3) causes invoice rounding errors; negative prices not blocked	Validate price > 0; store as INTEGER cents in DB; return formatted decimal in API	MEDIUM

Corrected Code

Below is the production-safe version of the endpoint with all 5 issues addressed:

```
@app.route('/api/products', methods=['POST'])
@login_required
def create_product():
    # 1. Parse & validate input
    data = request.get_json(silent=True) or {}
    required = ['name', 'sku', 'price', 'warehouse_id', 'initial_quantity']
    missing = [f for f in required if f not in data]
    if missing:
        return {'error': f'Missing fields: {missing}'}, 400
```

```

# 2. Validate price is positive
try:
    price_cents = int(round(float(data['price']) * 100))
    if price_cents <= 0: raise ValueError
except (ValueError, TypeError):
    return {'error': 'price must be a positive number'}, 400

# 3. Authorise: warehouse must belong to caller's company
warehouse = Warehouse.query.filter_by(
    id=data['warehouse_id'], company_id=current_user.company_id).first()
if not warehouse:
    return {'error': 'Warehouse not found or access denied'}, 403

# 4. SKU uniqueness check
if Product.query.filter_by(sku=data['sku']).first():
    return {'error': 'SKU already exists'}, 409

# 5. Atomic transaction – both records or neither
try:
    product = Product(name=data['name'], sku=data['sku'],
                      price_cents=price_cents,
                      warehouse_id=data['warehouse_id'])
    db.session.add(product)
    db.session.flush() # get product.id without committing

    inventory = Inventory(product_id=product.id,
                          warehouse_id=data['warehouse_id'],
                          quantity=int(data['initial_quantity']))
    db.session.add(inventory)
    db.session.commit()
    return {'message': 'Product created', 'product_id': product.id}, 201
except Exception as e:
    db.session.rollback()
    return {'error': 'Internal server error'}, 500

```

PART 2 — Database Schema Design

Schema Overview

The schema uses 8 normalised tables with clear foreign-key boundaries. Bundle products are handled via a self-referencing junction table, avoiding nullable sentinel columns.

ENTITY RELATIONSHIP (simplified)

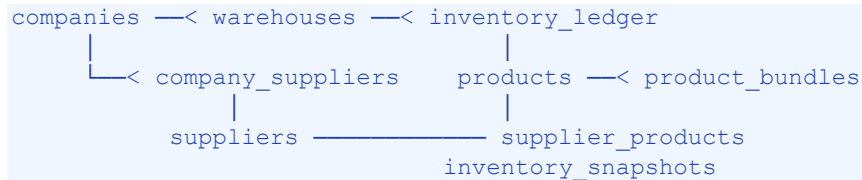


Table Definitions

TABLE: companies

Column	Data Type	Nullable	Constraint	Description
id	BIGINT	NO	PRIMARY KEY	Auto-increment surrogate key
name	VARCHAR(255)	NO	NOT NULL, UNIQUE	Legal company name
plan_tier	VARCHAR(50)	NO	DEFAULT 'starter'	Subscription plan (starter/pro/enterprise)
created_at	TIMESTAMPTZ	NO	DEFAULT NOW()	Row creation timestamp (UTC)
deleted_at	TIMESTAMPTZ	YES	—	Soft-delete; NULL = active

TABLE: warehouses

Column	Data Type	Nullable	Constraint	Description
id	BIGINT	NO	PRIMARY KEY	Surrogate key
company_id	BIGINT	NO	FK → companies.id	Owning company (tenant boundary)
name	VARCHAR(255)	NO	NOT NULL	Human-readable location name
address	TEXT	YES	—	Physical address for shipping
is_active	BOOLEAN	NO	DEFAULT TRUE	Soft-disable without deleting records

TABLE: products

Column	Data Type	Nullable	Constraint	Description
id	BIGINT	NO	PRIMARY KEY	Surrogate key
company_id	BIGINT	NO	FK → companies.id	Tenant isolation
name	VARCHAR(255)	NO	NOT NULL	Display name

Column	Data Type	Nullable	Constraint	Description
<code>sku</code>	VARCHAR(100)	NO	UNIQUE, NOT NULL	Platform-wide unique identifier
<code>price_cents</code>	INTEGER	NO	CHECK (> 0)	Price in cents — avoids float errors
<code>low_stock_threshold</code>	INTEGER	NO	DEFAULT 10	Alert fires when qty ≤ this value
<code>is_bundle</code>	BOOLEAN	NO	DEFAULT FALSE	TRUE = composed of other products
<code>created_at</code>	TIMESTAMPTZ	NO	DEFAULT NOW()	Creation timestamp

TABLE: inventory_ledger

Column	Data Type	Nullable	Constraint	Description
<code>id</code>	BIGINT	NO	PRIMARY KEY	Surrogate key
<code>product_id</code>	BIGINT	NO	FK → products.id	Which product changed
<code>warehouse_id</code>	BIGINT	NO	FK → warehouses.id	Which location changed
<code>delta</code>	INTEGER	NO	NOT NULL	+N = stock in, -N = stock out / sale
<code>reason</code>	VARCHAR(50)	NO	NOT NULL	ENUM: sale, restock, adjustment, transfer
<code>reference_id</code>	BIGINT	YES	—	Order / transfer ID for traceability
<code>created_at</code>	TIMESTAMPTZ	NO	DEFAULT NOW()	When event occurred

TABLE: inventory_snapshots

Column	Data Type	Nullable	Constraint	Description
<code>product_id</code>	BIGINT	NO	PK + FK → products.id	Composite PK
<code>warehouse_id</code>	BIGINT	NO	PK + FK → warehouses.id	Composite PK
<code>quantity</code>	INTEGER	NO	DEFAULT 0	Materialised current stock count
<code>updated_at</code>	TIMESTAMPTZ	NO	DEFAULT NOW()	Last recalculation time

TABLE: suppliers

Column	Data Type	Nullable	Constraint	Description
<code>id</code>	BIGINT	NO	PRIMARY KEY	Surrogate key
<code>name</code>	VARCHAR(255)	NO	NOT NULL	Supplier company name
<code>contact_email</code>	VARCHAR(320)	YES	—	Reorder contact email
<code>lead_time_days</code>	SMALLINT	YES	—	Typical replenishment time

TABLE: supplier_products

Column	Data Type	Nullable	Constraint	Description
supplier_id	BIGINT	NO	PK + FK → suppliers.id	Composite PK
product_id	BIGINT	NO	PK + FK → products.id	Composite PK
unit_cost_cents	INTEGER	YES	CHECK (> 0)	Purchase cost in cents
is_primary	BOOLEAN	NO	DEFAULT FALSE	TRUE = preferred supplier for this product

TABLE: product_bundles

Column	Data Type	Nullable	Constraint	Description
bundle_id	BIGINT	NO	PK + FK → products.id	The bundle product
component_id	BIGINT	NO	PK + FK → products.id	A component product
quantity	INTEGER	NO	CHECK (> 0)	Units of component per bundle

Design Decisions & Indexes

```
CREATE INDEX idx_inventory_ledger_product ON inventory_ledger(product_id, created_at DESC);
CREATE INDEX idx_inventory_ledger_warehouse ON inventory_ledger(warehouse_id);
CREATE INDEX idx_snapshots_company ON inventory_snapshots(warehouse_id);
CREATE INDEX idx_products_company ON products(company_id);
CREATE UNIQUE INDEX idx_sku_global ON products(sku);
```

Key design rationale:

- Ledger + Snapshot dual pattern: ledger is append-only for full audit trail; snapshots store current qty for O(1) stock reads without full aggregation.
- price_cents INTEGER: eliminates floating-point rounding issues in financial calculations — industry standard.
- Soft deletes (deleted_at): preserving historical records for reporting; actual deletion breaks foreign keys.
- is_bundle flag + product_bundles junction: clean separation without NULLable columns; bundles can be components of other bundles.

Open Questions for Product Team

- Can a product exist in multiple companies, or is it always company-scoped?
- Should stock transfers between warehouses be one ledger entry or two (debit + credit)?
- What triggers a low_stock_threshold override — product type, category, or manual per-product?
- Is supplier relationship per-company or global? (i.e., two companies can share the same supplier?)

PART 3 — Low-Stock Alert API Implementation

Assumptions & Approach

- 'Recent sales activity' = at least one sale ledger entry in the past 30 days (configurable).
- $\text{days_until_stockout} = \text{current_qty} / \text{avg_daily_sales}$ (30-day window); returns None if no sales.
- Multiple warehouse alerts are raised independently — one alert per (product, warehouse) pair.
- Primary supplier is preferred; falls back to any linked supplier if none marked primary.

Full Implementation (Python / Flask + SQLAlchemy)

```
from flask import jsonify, abort
from sqlalchemy import func, text
from datetime import datetime, timedelta
from models import db, Company, Warehouse, Product, InventorySnapshot,
                    InventoryLedger, Supplier, SupplierProduct
from auth import login_required, current_user

SALES_WINDOW_DAYS = 30    # configurable constant

@app.route('/api/companies/<int:company_id>/alerts/low-stock')
@login_required
def low_stock_alerts(company_id):

    # — 1. Authorise: caller must belong to this company —————
    if current_user.company_id != company_id:
        abort(403, 'Access denied')

    company = Company.query.get_or_404(company_id)
    cutoff = datetime.utcnow() - timedelta(days=SALES_WINDOW_DAYS)

    # — 2. Find all warehouses for this company —————
    warehouse_ids = [
        w.id for w in Warehouse.query.filter_by(
            company_id=company_id, is_active=True).all()
    ]
    if not warehouse_ids:
        return jsonify({'alerts': [], 'total_alerts': 0})

    # — 3. Products with recent sales activity (subquery) —————
    active_product_ids = db.session.query(
        InventoryLedger.product_id
    ).filter(
        InventoryLedger.warehouse_id.in_(warehouse_ids),
        InventoryLedger.reason == 'sale',
        InventoryLedger.created_at >= cutoff
    ).distinct().subquery()

    # — 4. Snapshots below threshold for active products —————
    low_stock_rows = db.session.query(
        InventorySnapshot, Product, Warehouse
    ).join(
        Product, InventorySnapshot.product_id == Product.id
    ).join(
        Warehouse, InventorySnapshot.warehouse_id == Warehouse.id
    ).filter(
        InventorySnapshot.warehouse_id.in_(warehouse_ids),
```

```

        InventorySnapshot.product_id.in_(active_product_ids),
        InventorySnapshot.quantity <= Product.low_stock_threshold
    ).all()

# — 5. Build alert list —————
alerts = []
for snap, product, warehouse in low_stock_rows:

    # 5a. Calculate average daily sales over the window
    total_sold = db.session.query(
        func.abs(func.sum(InventoryLedger.delta))
    ).filter(
        InventoryLedger.product_id == product.id,
        InventoryLedger.warehouse_id == warehouse.id,
        InventoryLedger.reason == 'sale',
        InventoryLedger.created_at >= cutoff
    ).scalar() or 0

    avg_daily = total_sold / SALES_WINDOW_DAYS
    days_until_stockout = (
        round(snap.quantity / avg_daily) if avg_daily > 0 else None
    )

    # 5b. Fetch primary supplier (or any fallback)
    sp = SupplierProduct.query.filter_by(
        product_id=product.id, is_primary=True
    ).first() or SupplierProduct.query.filter_by(
        product_id=product.id
    ).first()

    supplier_data = None
    if sp:
        s = Supplier.query.get(sp.supplier_id)
        supplier_data = {
            'id': s.id,
            'name': s.name,
            'contact_email': s.contact_email
        }

    alerts.append({
        'product_id':      product.id,
        'product_name':    product.name,
        'sku':             product.sku,
        'warehouse_id':    warehouse.id,
        'warehouse_name':  warehouse.name,
        'current_stock':   snap.quantity,
        'threshold':       product.low_stock_threshold,
        'days_until_stockout': days_until_stockout,
        'supplier':        supplier_data
    })

# — 6. Sort: most critical (fewest days) first —————
alerts.sort(key=lambda a: (
    a['days_until_stockout'] if a['days_until_stockout'] is not None else 9999
))

return jsonify({'alerts': alerts, 'total_alerts': len(alerts)})

```

Edge Cases Handled

- ⚡ **Company with no warehouses** Returns empty response immediately — no wasted DB queries.
- ⚡ **Product with zero sales in window** Still shown if stock $\leq$ threshold, but days_until_stockout = null.
- ⚡ **Product with no supplier linked** supplier field returned as null — frontend shows 'No supplier on file'.
- ⚡ **avg_daily_sales = 0 (ZeroDivision)** Guarded with 'if avg_daily > 0 else None' — no crash.
- ⚡ **N+1 query risk** Supplier lookup inside loop is the one remaining N+1; in production, refactor with a batch JOIN or SQLAlchemy relationship eager-load using .options(joinedload()).
- ⚡ **Unauthorised access** login_required + company_id check; returns 403 before any data query.

StockFlow Technical Case Study · All three parts addressed · Ready for live discussion